

DataLab 实验报告

姓名：穆思成 学号：2025200670

本实验在规定的操作符和操作次数内，完成 bits.c 中的 12 个函数。整数题主要用到补码、掩码和移位；浮点数题通过拆分符号位、阶码和尾数来实现转换与运算。

每题做法

1. bitAnd

用德摩根律把与运算改写成 $\sim(\sim x \mid \sim y)$ 。对每一位来说，只有两个输入都为 1 时，取反后的或运算结果才是 0，再取反就得到 1。

2. bitXor

异或要求两位不同。 $\sim(x \& y)$ 排除两位都是 1 的情况， $\sim(\sim x \& \sim y)$ 排除两位都是 0 的情况，把两者按位与起来即可：

```
 $\sim(x \& y) \& \sim(\sim x \& \sim y)$ 
```

3. sameSign

只比较符号位会把 0 和正数归为一类，所以还要用 $!x$ 区分零值。可以把 $(x \gg 31) \wedge !x$ 看成分类标记：负数为全 1，正数为 0，零为 1。两个标记异或后取逻辑非，就能判断是否属于同一类。

因此 (0, 0) 返回 1，零与任意非零数搭配都返回 0。

4. logTwo

结果等于最高位 1 的下标，也就是向下取整的 $\log_2(v)$ 。代码按 16、8、4、2、1 位逐步缩小范围。

例如，先判断 $v > 0xFFFF$ ：成立就说明最高位 1 在高 16 位，将 v 右移 16 位，并把 16 记入结果。之后对剩余部分重复这个过程。各次位移量占据结果的不同二进制位，所以可以用 $|$ 累计。比如 32 最终得到 5，31 得到 4。

5. byteSwap

先用 $n \ll 3$ 和 $m \ll 3$ 算出两个字节的偏移，右移后与 $0xFF$ 得到字节值 nb 、 mb 。再计算 $diff = nb \wedge mb$ ，把差异移回两个位置，与原数异或。

这样原来的 nb 会变成 $nb \wedge (nb \wedge mb) = mb$ ，另一个位置同理，其余位保持原值。 $n == m$ 时差异为 0，也能直接得到正确结果。

6. reverse

循环 32 次，每次取出 v 的最低位，接到 res 左移一位后的末尾，再将 v 右移。最先取出的最低位经过后续移位，最后到达结果的最高位，从而完成反转。

这里使用 `unsigned`，右移时高位补 0。循环固定执行 32 次，保证原数高位的零也参与反转。

7. logicalShift

先算术右移，再用掩码清除高位补入的符号位：

```
(x >> n) & ~(((1 << 31) >> n) << 1)
```

按实验环境的 32 位补码和算术右移行为，掩码的高 n 位为 0，其余位为 1。 $n == 0$ 时掩码全为 1， $n == 31$ 时只保留最低位；整个表达式也避免了直接写出移动 32 位的情况。

8. leftBitCount

先对 x 取反，把“统计开头连续的 1”转成“统计开头连续的 0”。然后依次检查高 16、8、4、2、1 位是否全为 0；如果是，就增加计数，并将已经统计过的部分左移出去。

这些步骤最多累计到 31。原数为 -1 时，取反后全为 0，需要得到 32，因此保存取反后的初值 y ，最后用 $n + !y$ 补上这种情况。

9. float_i2f

零直接返回 0。对非零数先保存符号位，负数用 $\sim y + 1$ 得到幅值的位模式，再不断左移，直到最高位为 1。阶码从 $158 = 127 + 31$ 开始，每左移一次减 1。

对齐后，最高位的 1 是隐藏位，接下来的 23 位作为尾数，低 8 位用于判断舍入：

- 低 8 位大于 $0x80$ ，超过半个舍入单位，尾数加 1。
- 低 8 位等于 $0x80$ ，且保留部分的最低位为 1，即 $(y \& 0x1FF) == 0x180$ ，也加 1，使结果舍入到偶数。
- 其余情况不进位。

最后用加法拼接符号位、阶码和尾数。若尾数因舍入溢出，进位会自然进入阶码。对于 INT_MIN ，幅值位模式为 $0x80000000$ ，无需继续左移，结果表示 -2^{31} 。

10. floatScale2

按阶码分类处理：

- 阶码全为 1：输入是 NaN 或无穷大，返回原值。
- 阶码为 0：保留符号位，其余部分左移一位，完成非规格化数乘 2；若产生进位，也会自然进入规格化范围。正负零同样适用。
- 阶码为 254：乘 2 后溢出，返回同符号的无穷大，并清空尾数。
- 其余规格化数：直接加 $0x00800000$ ，使阶码增加 1。

11. float64_f2i

从高 32 位 $uf2$ 中提取阶码，减去偏置 1023，得到实际指数 e 。 $e < 0$ 时绝对值小于 1，向零取整后返回 0； $e \geq 31$ 时返回 $0x80000000$ 。合法的 -2^{31} 恰好也是这个位模式，NaN 和无穷大也会进入后一支。

剩下的情况先将高 20 位尾数与隐藏的 1 拼成 $high$ ：

- $e \leq 20$ ：整数部分完全位于 $high$ 中，右移 $20 - e$ 位即可。
- $e > 20$ ：将 $high$ 左移 $e - 20$ 位，再从低 32 位 $uf1$ 中补入所需位，即 $uf1 \gg (52 - e)$ 。

移位直接丢弃小数部分，最后根据符号位取补码，得到向零取整的结果。

12. floatPower2

2^x 的位模式由指数范围决定：

x 的范围	处理方式
$x < -149$	小于最小非规格化正数，返回 0
$-149 \leq x < -126$	非规格化数，返回 $1 \ll (x + 149)$
$-126 \leq x \leq 127$	规格化数，尾数为 0，返回 $(x + 127) \ll 23$
$x > 127$	上溢，返回正无穷 $0x7F800000$

测试结果

运行 `python3 test.py -V`，编译无警告，12 个函数的正确性和操作限制检查全部通过，脚本输出 Total points: 37。

函数	得分	操作数 / 上限
bitAnd	1/1	4/7
bitXor	1/1	7/7
samesign	2/2	8/12
logtwo	4/4	18/25
byteSwap	4/4	11/17
reverse	3/3	5/30
logicalShift	3/3	6/20
leftBitCount	4/4	26/50
float_i2f	4/4	19/30
floatScale2	4/4	10/30
float64_f2i	3/3	18/60
floatPower2	4/4	9/30

得分和操作数均以 `test.py -V` 的实际输出为准；操作数统计的是代码中的运算符数量，不是循环运行时的执行次数。

实验小结

整数题的关键是把条件转成位标记，再用掩码或异或完成处理。浮点数题更需要注意边界，尤其是舍入进位、非规格化数与规格化数的交界，以及整数转换时的范围判断。